

Formal Methods and Specification (SS 2025/26)

Lecture 9:

Symbolic Program Execution and Bounded Software Model Checking

Stefan Ratschan

Czech Technical University in Prague



Today's Program

Up to now: techniques for ensuring **correctness**

Today: finding **bugs** ... in a completely automated fashion

Prelude: Validity vs. Satisfiability

For example: for real numbers

$$\not\models x \geq y?$$

or, equivalently

$$\neg x \geq y \text{ that is } x < y \text{ satisfiable}$$

satisfying assignment and counter-example, e.g. $\{x \mapsto 3, y \mapsto 4\}$

In general:

$$\not\models \phi$$

iff

formula $\neg\phi$ is satisfiable

Intuition: satisfiability = solvability

$$\text{counter-example to } \models \phi \iff \text{assignment satisfying } \neg\phi$$

Symbolic Program Execution

Technique that

- ▶ already **helped everybody**, who has ever used Microsoft Windows and that
- ▶ helped Microsoft to **save millions** of dollars, and
- ▶ that Microsoft has been running on **> 100 computers since 2008**.

[Godefroid et al., 2012]

Symbolic Program Execution

We want to **test** programs as **completely** as possible.

For example, as complete coverage of code with test cases as possible
(viz coverage criteria)

Execution path: sequence of program lines.

Main question:

How to **cover** as many **execution paths** as possible with test cases?

Example:

```
1: while  $x \geq 0$  do
2:   input  $x$ 
3:   if  $2x - 1 \geq 3$  then
4:      $x \leftarrow x - 2$ 
5:   else
6:      $x \leftarrow x - 1$ 
```

How to follow the execution path 1, 2, 3, 4, 1, 2, 3, 6?

$$x \geq 0 \wedge 2x - 1 \geq 3 \wedge x = x - 2 \wedge x \geq 0 \wedge 2x - 1 < 3$$

Initial value and inputs for x may differ!

$$x_1 \geq 0 \wedge 2x_2 - 1 \geq 3 \wedge x_3 = x_2 - 2 \wedge x_3 \geq 0 \wedge 2x_4 - 1 < 3$$

Conjunction of all assignments and conditions along the path + variables indices

A solver can then find a test case or prove that none exists (demo).

Partial Formalization

Program P :

```
1: while  $x \geq 0$  do  
2:   input  $x$   
3:   if  $2x - 1 \geq 3$  then  
4:      $x \leftarrow x - 2$   
5:   else  
6:      $x \leftarrow x - 1$ 
```

Corresponding basic path

$BP_{1,2,3,4,1,2,3,6}(P)$

```
1: assume  $x \geq 0$   
2: input  $x$   
3: assume  $2x - 1 \geq 3$   
4:  $x \leftarrow x - 2$   
1: assume  $x \geq 0$   
2: input  $x$   
3: assume  $\neg 2x - 1 \geq 3$   
  @  $\perp$ 
```

The program P can follow the execution path 1, 2, 3, 4, 1, 2, 3, 6
iff

the basic path $BP_{1,2,3,4,1,2,3,6}(P)$ is **incorrect**

iff

not $\models VC(BP_{1,2,3,4,1,2,3,6}(P))$

iff

$\neg VC(BP_{1,2,3,4,1,2,3,6}(P))$ is satisfiable

Formalization

Program P :

```
1: while  $x \geq 0$  do  
2:   input  $x$   
3:   if  $2x - 1 \geq 3$  then  
4:      $x \leftarrow x - 2$   
5:   else  
6:      $x \leftarrow x - 1$ 
```

Corresponding basic path
 $BP_{1,2,3,4,1,2,3,6}(P)$

```
1: assume  $x \geq 0$   
2: input  $x$   
3: assume  $2x - 1 \geq 3$   
4:  $x \leftarrow x - 2$   
1: assume  $x \geq 0$   
2: input  $x$   
3: assume  $\neg 2x - 1 \geq 3$   
   @  $\perp$ 
```

$\neg VC(BP_{1,2,3,4,1,2,3,6}(P))$

is equivalent to

$\neg[[x_1 \geq 0 \wedge 2x_2 - 1 \geq 3 \wedge x_3 = x_2 - 2 \wedge x_3 \geq 0 \wedge 2x_4 - 1 < 3] \Rightarrow \perp]$

which is equivalent to

$x_1 \geq 0 \wedge 2x_2 - 1 \geq 3 \wedge x_3 = x_2 - 2 \wedge x_3 \geq 0 \wedge 2x_4 - 1 < 3$

If **satisfiable**, then satisfying assignment results in a **test case**

Summary

A program P can follow an execution path $l_1, \dots, l_n$
iff
 $\neg VC(BP_{l_1, \dots, l_n}(P))$ is satisfiable

Here: $BP_{l_1, \dots, l_n}(P)$ is the basic path that

- ▶ consists of the program lines $l_1, \dots, l_n$,
with all control structures replaced by **assume**-assertions
that must be satisfied when following $l_1, \dots, l_n$, and that
- ▶ has $@ \perp$ as its final assertion.

Observation: $\neg VC(BP_{l_1, \dots, l_n}(P))$ can be written equivalently as a conjunction

Notation: $SE(BP_{l_1, \dots, l_n}(P))$

see also “error condition” at the end of lecture 4

Practical Usage of Symbolic Execution

Main question:

How to **cover** with test cases
as many **execution paths** as possible?

Method:

- ▶ Choose a set of execution paths
- ▶ For every execution path $l_1, \dots, l_n$,
find the corresponding test case by
computing a satisfiable assignment of $SE(BP_{l_1, \dots, l_n}(P))$

Problems:

- ▶ **undecidable** theories (e.g., due to non-linear operations on integers)
- ▶ usage of **external functions** with unknown source code
(e.g., from the operating system)

Example

Basic path:

```
...  
x  $\leftarrow$  2x + 1  
r  $\leftarrow$  ext(x)  
assume r > 7  
y  $\leftarrow$  xy + z  
assume y > 3  
...
```

Condition for regular execution:

```
x2 = 2x1 + 1  $\wedge$   
r = ext(x2)  $\wedge$   
r > 7  $\wedge$   
y2 = x2y1 + z  $\wedge$   
y2 > 3
```

where ext() can, for example read a sensor in a nuclear power plant, call a function on super-computer, install some software on 2000 computers etc.

Hence **no solver** can handle this. What to do? approximation!

Handling Complex Functions: Over-Approximation

$$x_2 = 2x_1 + 1 \wedge r = \text{ext}(x_2) \wedge r > 7 \wedge y_2 = x_2y_1 + z \wedge y_2 > 3$$

$$x_2 = 2x_1 + 1 \wedge \top \wedge r > 7 \wedge y_2 = x_2y_1 + z \wedge y_2 > 3$$

What does this mean? weaker constraint

Satisfying assignment (i.e., a solution) is

not necessarily a satisfying assignment of the original formula.

In verification, where unsatisfiability means absence of an error,
this is what we want.

In symbolic execution: probably does not follow the wanted lines

Hence over-approximation is not what we want here

Handling Complex Functions: Under-Approximation

$$x_2 = 2x_1 + 1 \wedge r = \text{ext}(x_2) \wedge r > 7 \wedge y_2 = x_2y_1 + z \wedge y_2 > 3$$

$$x_2 = 2x_1 + 1 \wedge \perp \wedge r > 7 \wedge y_2 = x_2y_1 + z \wedge y_2 > 3$$

unsatisfiable formula, better under-approximation?

We can execute the function `ext` for arbitrary inputs!

We compute the result for random input, e.g., $x_2 = 0$, and $r = 0$.

$$x_2 = 2x_1 + 1 \wedge x_2 = 0 \wedge r = 0 \wedge r > 7 \wedge y_2 = x_2y_1 + z \wedge y_2 > 3$$

Problem?

Then $x_2 = 2x_1 + 1$ does not have a solution (if x_1 is an integer)

First we search for a solution of $x_2 = 2x_1 + 1$, then we compute `ext`(x_2) and so on

Example

$x \leftarrow 2x + 1$

$r \leftarrow \text{ext}(x)$

assume $r > 7$

$y \leftarrow xy + z$

assume $y > 3$ **then**

...

$x_2 = 2x_1 + 1 \wedge$

$x_2 = 5 \wedge r = 13 \wedge$

$r > 7 \wedge$

$y_2 = x_2y_1 + z \wedge$

$y_2 > 3$

1. We extract a formula from the first program line
2. We arrive at the external function $\text{ext}()$, for its execution we need input (x_2)
3. We solve $x_2 = 2x_1 + 1$ e.g., $x_1 \mapsto 2, x_2 \mapsto 5$
4. We execute $\text{ext}(5)$, the result can, for example, be 13
5. Instead of $r = \text{ext}(x_2)$ we add $x_2 = 5, r = 13$ to the formula.
6. The resulting formula is in a solvable class, so we can continue.

Formalization

Input: basic path P (may call external functions)

Output: formula ϕ such that every assignment satisfying ϕ corresponds to a regular program execution of P

Assumption: external function calls do not have side effects

Implementation: $\text{trans}(L, \top)$, where L is a list of pairs (c, c') where

- ▶ c corresponds to the program lines in P , and
- ▶ c' is the corresponding atomic sub-formula of $SE(P)$

$\text{trans}(\langle \rangle, \phi) = \phi$

$\text{trans}(\text{cons}((v \leftarrow \text{ext}(x), c'), r), \phi) = \text{trans}(r, x = \nu_x \wedge v = \nu_v \wedge \phi)$, where

- ▶ ν_x is the value of x in an assignment satisfying ϕ , and
- ▶ ν_v is the result of executing $\text{ext}(\nu_x)$.

$\text{trans}(\text{cons}((c, c'), r), \phi) = \text{trans}(r, c' \wedge \phi)$

Dynamic Test Generation/Concolic Testing

Extension that allows side effects

Execute the program both **concretely** and **symbolically**.

Create the logical formula $SE(BP_{l_1, \dots, l_n}(P))$ **line by line**.

As soon as formula creation

arrives at a function that the solver cannot handle,

let the program run until we receive the result of the function.

If we need user inputs for this,

we compute them by solving the symbolic formula

(of the part corresponding to the program before the external function)

Use the result of the concrete execution to

replace the external function call by corresponding equalities

Completeness

Does the method always find a test case following a given path, if it exists?

(of course) not, we only solve the formula approximately:

$$x \leftarrow 2x + 1$$

$$r \leftarrow \text{ext}(x)$$

$$\textbf{assume } r > 7$$

$$y \leftarrow xy + z$$

$$\textbf{assume } y > 3$$

...

$$x_2 = 2x_1 + 1 \wedge$$

$$x_2 = 5 \wedge r = 6 \wedge$$

$$r > 7 \wedge$$

$$y_2 = x_2 y_1 + z \wedge$$

$$y_2 > 3$$

Assume that the result of executing $\text{ext}(5)$ is 6, and of $\text{ext}(7)$ it is 13.

Due to our choice of a solution of $x_2 = 2x_1 + 1$ we add $x_2 = 5, r = 6$, and the resulting formula does not have a solution.

Symbolic Execution for Test Case Generation

Cover **tree** of paths

(either if or else, either we stay in loop, or we leave it)

... up to certain depth

formula is built and checked incrementally

```
x ← 2x + 1
r ← ext(x)
if r > 7 then
  y ← xy + z
  if y > 3 then
    ...
  else
    ...
else
  ...
```

```
x2 = 2x1 + 1 ∧
x2 = 5 ∧ r = 13 ∧
r > 7 ∧
y2 = x2y1 + z ∧
¬y2 > 3
```

Path Explosion Problem

```
@  $\forall i \in \{1, \dots, 1000\} . a[i] \in \{0, 1\}$   
for  $i \leftarrow 1$  to 1000 do  
    if  $a[i] = 0$  then  
        ...  
    else  
        ...
```

How many execution paths? 2^{1000}

price for avoiding loop invariants

The **coverage** of the full tree of execution paths
up to a practically relevant depth is often **unrealistic**.

Possible solutions:

- ▶ choice of paths
- ▶ merging paths

SAGE: Whitebox Fuzzing [Godefroid et al., 2012]

Instead of depth-first search, **local** search:

Using test cases from a different source,
cover **neighboring paths** by negating the individual conditions.

Technological issues (e.g., works on machine code)

Regularly finds security bugs (e.g., buffer overflow),
which **saves security updates**, that Microsoft would have to
send to millions of computers, worldwide.

<https://en.wikipedia.org/wiki/OneFuzz>

Symbolic Execution: Tools and Literature

Many tools available.

Some for specific programming languages,
some for LLVM intermediate representation.

see Wikipedia “Symbolic execution”, “Concolic testing”

Survey articles: [Cadaru and Sen, 2013], [Baldoni et al., 2018]

MI-TES: symbolic execution of timed automata
(program line $\sim$ location of timed automaton)

Bounded Software Model Checking

Method for **automatic** program **correctness proofs** that

- ▶ is used by Amazon Web Services to check memory safety of boot code at data centers [Cook et al., 2020]
- ▶ found bugs in a basic software package necessary for the internet (Internet Systems Consortium) [Cho et al., 2013]
- ▶ ...

Loop Unrolling: Example:

```
 $s \leftarrow 0$   
 $i \leftarrow 0$   
while  $i \leq n$  do  
     $s \leftarrow s + a[i]$   
     $i \leftarrow i + 1$   
 $i \leftarrow 0$   
while  $i \leq n$  do  
     $s \leftarrow s - a[i]$   
     $i \leftarrow i + 1$   
@  $s \leq 0$ 
```

Loop Unrolling: Example:

$s \leftarrow 0$
 $i \leftarrow 0$
assume $\neg i \leq n$

$i \leftarrow 0$
assume $\neg i \leq n$

@ $s \leq 0$

Loop Unrolling: Example:

```
 $s \leftarrow 0$   
 $i \leftarrow 0$   
if  $i \leq n$  then  
     $s \leftarrow s + a[i]$   
     $i \leftarrow i + 1$   
    assume  $\neg i \leq n$   
 $i \leftarrow 0$   
if  $i \leq n$  then  
     $s \leftarrow s - a[i]$   
     $i \leftarrow i + 1$   
    assume  $\neg i \leq n$   
@  $s \leq 0$ 
```

Loop Unrolling: Example:

$s \leftarrow 0; i \leftarrow 0$

if $i \leq n$ **then**

$s \leftarrow s + a[i]$

$i \leftarrow i + 1$

if $i \leq n$ **then**

$s \leftarrow s + a[i]$

$i \leftarrow i + 1$

assume $\neg i \leq n$

$i \leftarrow 0$

if $i \leq n$ **then**

$s \leftarrow s - a[i]$

$i \leftarrow i + 1$

if $i \leq n$ **then**

$s \leftarrow s - a[i]$

$i \leftarrow i + 1$

assume $\neg i \leq n$

@ $s \leq 0$

Comparison: Correctness of
original vs. changed program?

Loop Unrolling

Problem: for n **if-then** statements, 2^n basic paths

Can we translate **if** statements into logical formulas
without creating exponentially many basic paths?

SSA with **if**

$s \leftarrow 0$

$i \leftarrow 0$

if $i \leq n$ **then**

$s \leftarrow s + a[i]$

$i \leftarrow i + 1$

if $i \leq n$ **then**

$s \leftarrow s + a[i]$

$i \leftarrow i + 1$

@ $s \leq 0$

SSA with **if**

```
s ← 0
i ← 0
if i ≤ n then
    s1 ← s + a[i]
    i1 ← i + 1
    if i1 ≤ n then
        s2 ← s1 + a[i1]
        i2 ← i1 + 1
@ s? ≤ 0
```

```
s ← 0
i ← 0
γ1 ← [i ≤ n]
s1 ← s + a[i]
i1 ← i + 1
γ2 ← [i1 ≤ n]
s2 ← s1 + a[i1]
s3 ← γ2 ? s2 : s1
s4 ← γ1 ? s3 : s
@ s4 ≤ 0
```

Here:

- ▶ $\gamma_i \in \{\top, \perp\}$, $i \in \mathbb{N}$
- ▶ $[\phi] := \begin{cases} \top, & \text{if } \phi \text{ is true for the current values of program variables, and} \\ \perp, & \text{otherwise} \end{cases}$
- ▶ $?$: behaves in the same way as the corresponding C-construct

SSA with **if-then-else**

if $p(x)$ **then**

$k \leftarrow 0$

else

$k \leftarrow 1$

@ $k \geq 0$

$\gamma_1 \leftarrow [p(x)]$

$k_1 \leftarrow 0$

$k_2 \leftarrow 1$

$k_3 \leftarrow \gamma_1 ? k_1 : k_2$

@ $k_3 \geq 0$

is correct

iff

$$\models [\gamma_1 \Leftrightarrow p(x) \wedge k_1 = 0 \wedge k_2 = 1 \wedge k_3 = \gamma_1 ? k_1 : k_2] \Rightarrow k_3 \geq 0$$

where $s = \phi ? t_1 : t_2$ stands for

$$[\phi \Rightarrow s = t_1] \wedge [\neg \phi \Rightarrow s = t_2]$$

Assertions Within Program

if $p(x)$ **then**

$k \leftarrow 0$

@ $k = 0$

else

$k \leftarrow 1$

@ $k \geq 0$

$\gamma \leftarrow [p(x)]$

$k_1 \leftarrow 0$

$k_2 \leftarrow 1$

$k_3 \leftarrow \gamma ? k_1 : k_2$

@ $[\gamma \Rightarrow k_1 = 0] \wedge k_3 \geq 0$

is correct

iff

$\models [\gamma_1 \Leftrightarrow p(x) \wedge k_1 = 0 \wedge k_2 = 1 \wedge k_3 = \gamma_1 ? k_1 : k_2] \Rightarrow$

$[[\gamma \Rightarrow k_1 = 0] \wedge k_3 \geq 0]$

Bounded Software Model Checking

1. Unroll loops a certain number of times
(the resulting program does not contain loops any more)
2. Translate to SSA
(each program variable corresponds to precisely one value)
3. Translate SSA to a logical formula
4. Check the resulting formula

$$BMC_P(n) := F(SSA(unroll_P(n)))$$

Bounded Program Correctness

If we only use data types from decidable theories
we can check $BMC_P(n)$ **automatically**.

If $\not\models BMC_P(n)$, then $\neg BMC_P(n)$ is satisfiable

The satisfying assignments represents variable values leading to a bug

This is called **counter-example**, **error trace**

If $\models BMC_P(n)$ then we did not find a bug.

This does **not** mean that the program is correct!
(finite number of loop unrollings)

Bounded Verification

Correctness within $0, 1, 2, \dots$ loop unrollings:

$BMC(0)$

$BMC(1)$

$BMC(2)$

$BMC(3)$

$\dots$

For $i = 0, 1, \dots$,

if $BMC(i)$ finds a bug, then $BMC(i + 1)$, as well, since
 $BMC(i + 1)$ implies $BMC(i)$

For simplicity, slightly **different** definition than in MI-TES

CBMC Demo

<http://www.cprover.org/cbmc/>

Original article [Clarke et al., 2004]

Today: more and more usage of BMC in industry: <http://www.btc-es.de>

Consequences

Every bug occurs after a certain finite number of steps.

Due to this, counter-examples always have a **certain length**

So we can **always find** them, if we have enough time:

for every incorrect program P
there is a k s.t. $\neg BMC_P(k)$

Hence: Finding errors in programs with data types in decidable theories
is **semi-decidable**.

In practice: Programs can do a huge number of steps,
and so we have to check $BMC_P(n)$ for huge n .

But: in **certain applications** that usually does **not** happen

For examples: **embedded systems**:

Reaction to a certain event may take only a short amount of time

Application: Equivalence Checking

```
function foo(x)
function foo_optimized(x)

input x
assert foo(x)=foo_optimized(x)
```

Demo: `cbmc equiv.c`

Further Application: Combination with Testing

For software in safety critical applications,
there are standards that require **completeness** of **tests**
according to a certain criteria.

Usually those criteria require that tests the **cover** program **code**
in a certain sense (*coverage criteria*).

For example: Tests have to execute **each program line** at least **once**.

Problem: How to find a test that executes line l ?

Check $BMC_{P_l}(1), BMC_{P_l}(2), \dots$, where
 P_l is the original program with line l replaced by $@ \perp$

European Train Control System (ETCS) [Angeletti et al., 2010]

Conclusion

Two techniques that are used in a completely **automated** fashion:

- ▶ symbolic program execution
- ▶ bounded software model checking

Literature I

- Damiano Angeletti, Enrico Giunchiglia, Massimo Narizzano, Alessandra Puddu, and Salvatore Sabina. Using bounded model checking for coverage analysis of safety-critical software in an industrial setting. *Journal of Automated Reasoning*, 45:397–414, 2010. ISSN 0168-7433.
- Roberto Baldoni, Emilio Coppa, Daniele Cono D'elia, Camil Demetrescu, and Irene Finocchi. A survey of symbolic execution techniques. *ACM Comput. Surv.*, 51(3):50:1–50:39, May 2018. ISSN 0360-0300.
- Cristian Cadar and Koushik Sen. Symbolic execution for software testing: Three decades later. *Commun. ACM*, 56(2):82–90, February 2013. ISSN 0001-0782. doi: 10.1145/2408776.2408795.
- Chia Yuan Cho, Vijay D'Silva, and Dawn Song. Blitz: Compositional bounded model checking for real-world programs. In *Automated Software Engineering (ASE), 2013 IEEE/ACM 28th International Conference on*, pages 136–146. IEEE, 2013.

Literature II

Edmund Clarke, Daniel Kroening, and Flavio Lerda. A tool for checking ANSI-C programs. In Kurt Jensen and Andreas Podelski, editors, *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2004)*, volume 2988 of *Lecture Notes in Computer Science*, pages 168–176. Springer, 2004. ISBN 3-540-21299-X.

Byron Cook, Kareem Khazem, Daniel Kroening, Serdar Tasiran, Michael Tautschnig, and Mark R Tuttle. Model checking boot code from AWS data centers. *Formal Methods in System Design*, pages 1–19, 2020.

Patrice Godefroid, Michael Y. Levin, and David Molnar. SAGE: whitebox fuzzing for security testing. *Commun. ACM*, 55(3):40–44, March 2012. doi: 10.1145/2093548.2093564.